

Buổi 3

Tìm kiếm nhị phân và nhị phân trên đáp án, bản Python

Kiến thức, mười bài tập và mười code mẫu

Ngày 23 tháng 9 năm 2026

TS. Đỗ Phúc Hảo

Tài liệu một file cho buổi chặt nhị phân, bản Python. Gồm bốn phần: kiến thức, mười đề bài, gợi ý và lời giải, và mười code mẫu đầy đủ.

Đề bài và bộ test dùng chung với bản C++ ở `../../topics/03-binary-search/`. Cùng một đề, cùng một bộ test, khác ngôn ngữ.

Buổi này là buổi mà hai ngôn ngữ **khác nhau ít nhất về thuật toán và nhiều nhất về danh sách phạm bẫy**. Thuật toán chặt nhị phân không đổi một dòng nào khi sang Python. Nhưng bốn trong tám phạm bẫy mà bản C++ liệt kê đều là phạm bẫy **tràn số**, và số nguyên Python không tràn, nên bốn cái đó biến mất sạch. Đổi lại Python cho bạn `bisect` chạy ở mức C, và mở ra một câu hỏi mới mà bản C++ không có: **khi nào gọi thư viện được, khi nào phải tự viết vòng lặp**.

1. Mục lục

Phần	Nội dung
I	Kiến thức, mười ba mục
II	Mười đề bài A tới J
III	Gợi ý phân tầng và lời giải từng bài
IV	Mười code mẫu đầy đủ

Phần I. Kiến thức

Mục tiêu: nhận ra một lớp bài rất rộng mà thoát nhìn chẳng liên quan gì tới tìm kiếm, nhưng lại giải được bằng cách **đoán đáp án rồi kiểm tra**. Đây là kỹ thuật cho tỷ lệ điểm cao nhất so với công sức học ở ICPC, vì nó biến bài tối ưu khó thành bài kiểm tra dễ.

Mục	Nội dung	Bài luyện
1	bisect của Python, và nó trả lời câu hỏi nào	A
2	Khi nào bisect vô dụng	C tới J
3	Viết tay vòng chặt cho đúng biên	A
4	Hai mẫu chuẩn và bẫy làm tròn	B, D, E
5	Chặt trên đáp án và điều kiện đơn điệu	C tới J
6	Hàm kiểm thường là tham lam	C, D, J
7	Chọn cận trên cho đúng	B, G
8	Chia lấy trần, và vì sao // của Python giúp bạn	F, J
9	Bốn cạm bẫy của bản C++ biến mất	B, E, F, G, I
10	Thoát sớm trong hàm kiểm: tám trên tám đều chậm đi	C tới J
11	Chặt nhị phân trên số thực	không có bài
12	Cạm bẫy Python ngoài phần thuật toán	tất cả
13	Bộ khung tự hỏi khi gặp đề mới	tất cả

1. Module bisect, và nó trả lời câu hỏi nào

Python có sẵn bisect trong thư viện chuẩn, viết bằng C. Ba hàm đáng nhớ:

Hàm	Trả về	Tương đương C++
bisect_left(a, x)	số phần tử nhỏ hơn thật sự x	lower_bound
bisect_right(a, x)	số phần tử không lớn hơn x	upper_bound
insort(a, x)	chèn x giữ mảng vẫn sắp	không có sẵn

Cả ba đòi mảng **đã sắp không giảm**, và không có hàm nào kiểm hộ bạn điều đó. Truyền vào một mảng chưa sắp thì bạn nhận một con số, không nhận một lời báo lỗi.

Hai hàm đầu khác nhau đúng ở một dấu bằng, và đó là lỗi hay gặp nhất khi mới dùng:

```
a = [1, 3, 3, 5, 7]
bisect_left(a, 3)    # 1, so phan tu nho hon 3
bisect_right(a, 3)   # 3, so phan tu khong lon hon 3
bisect_right(a, 3) - bisect_left(a, 3) # 2, so lan 3 xuat hien
```

Nhanh tới mức nào. Đo trên test lớn nhất của bài A, $2 \cdot 10^5$ truy vấn trên dãy $2 \cdot 10^5$ phần tử:

Cách viết	Thời gian
bisect_left	109 ms
vòng chặt tự viết	264 ms

Nhanh hơn 2,4 lần. Toàn bộ khoản chênh đó là chi phí của trình thông dịch: vòng tự viết chạy khoảng 3,6 triệu lượt lặp trong Python, còn `bisect_left` chạy đúng chừng ấy lượt trong C.

Một cái bẫy của phiên bản. Tham số `key=` của `bisect_left` chỉ có từ Python 3.10. Máy dựng nhánh này chạy **Python 3.9.6**, nên `bisect_left(a, x, key=f)` ở đây ném `TypeError`. Muốn chặt theo một khoá dẫn xuất trên 3.9 thì phải dựng sẵn một mảng khoá riêng, hoặc tự viết vòng lặp.

2. Khi nào bisect vô dụng

Đây là mục quan trọng nhất của cả buổi, và nó là chỗ bản Python phải nói khác bản C++.

`bisect` chặt trên **một mảng có thật trong bộ nhớ**. Nhưng chín trong mười bài của buổi này không có mảng nào để chặt cả. Chúng chặt trên một **khoảng đáp án**:

Bài	Chặt trên cái gì	Có mảng không
A	mảng <code>a</code> đã sắp	có, nhưng đề cấm gọi thư viện
B	khoảng từ 0 tới $2 \cdot 10^9$	không
C	khoảng từ phần tử lớn nhất tới tổng cả dãy	không
D	khoảng khoảng cách có thể	không
E	khoảng độ cao lưỡi cửa	không
F	khoảng tốc độ ăn	không
G	khoảng số phút	không
H	khoảng giá trị trong bảng nhân	không
I	khoảng khoảng cách	không
J	khoảng độ dài đoạn lớn nhất	không

Dãy gồm mọi số nguyên từ 0 tới $2 \cdot 10^9$ không tồn tại trong bộ nhớ, và cũng không nên tồn tại. Thứ tồn tại là một **hàm đơn điệu** `ok(X)` trả về đúng hoặc sai. Chặt trên nó thì phải tự viết vòng lặp.

Nên câu trả lời gọn là thế này. `bisect` cho bạn chặt trên **dữ liệu**. Buổi này chủ yếu chặt trên **đáp án**. Hai chuyện khác nhau, và chỉ chuyện thứ nhất có hàm dựng sẵn.

Bài A cấm `bisect` chính vì vậy: nếu buổi học bắt đầu bằng một lời gọi thư viện thì chín bài sau không ai viết nổi.

Và cả khi có mảng, thư viện chưa chắc thắng. Bài I có mảng đã sắp, và hàm đếm cặp viết được bằng `bisect_left` một lần cho mỗi phần tử. Đo trên test lớn nhất:

Cách đếm cặp	Thời gian
hai con trỏ, Python thuần	523 ms
<code>bisect_left</code> , một lời gọi mỗi phần tử	1.143 ms

Bản gọi thư viện **chậm hơn 2,2 lần**. Quy tắc rút ra được: một hàm C chỉ thắng khi **một lời gọi của nó thay cho nhiều bước thông dịch**. Ở bài A, một lời gọi thay cho 18 bước, nên nó thắng. Ở bài I, một lời gọi thay cho đúng một bước trượt con trỏ, nên chi phí gọi hàm ăn hết phần lợi.

3. Viết tay vòng chặt cho đúng biên

Mẫu dùng chung cho cả buổi, với bất biến nửa mở `[lo, hi)`:

```
lo = 0; hi = n          # the answer lives in [lo, hi]
while lo < hi:
    mid = (lo + hi) >> 1
```

```

if a[mid] < x:
    lo = mid + 1      # mid is not the answer, discard it
else:
    hi = mid          # mid may still be the answer, keep it
# lo == hi == number of elements strictly less than x

```

Ba điều làm mẫu này an toàn, và điều thứ nhất **khác bản C++**:

- **Không cần công thức chống tràn.** Viết $(lo + hi) \gg 1$ là đủ. Bản C++ bắt viết $lo + (hi - lo) / 2$ để tránh tràn khi hai biên cùng lớn. Số nguyên Python không tràn, nên lý do đó không tồn tại, và lỗi viết phòng thủ ấy còn tốn thời gian. Đo trên bài A:

Cách tính điểm giữa	Thời gian
$(lo + hi) \gg 1$	248 ms
$(lo + hi) // 2$	262 ms
$lo + (hi - lo) // 2$	298 ms

Chép nguyên thói quen C++ sang đây là trả 20 phần trăm thời gian cho một hiểm hoạ đã không còn.

- **Hai nhánh không đối xứng.** Nhánh loại bỏ dùng $mid + 1$, nhánh giữ lại dùng mid . Viết $hi = mid - 1$ ở nhánh thứ hai là **vứt mất chính đáp án**: khi $a[mid]$ đúng bằng x , vị trí mid là đáp án, và bạn vừa loại nó. Nếu cả hai nhánh cùng dùng mid thì khi lo và hi cách nhau một đơn vị, mid bằng lo và vòng lặp không bao giờ thoát.
- **Vòng lặp kết thúc khi cửa sổ rỗng**, nên lo cuối cùng luôn là đáp án, không cần kiểm lại.

Một chú ý riêng của Python về dấu. $//$ và $\gg$ của Python làm tròn về phía âm vô cùng, còn phép chia nguyên của C++ cắt về phía không. Với lo và hi không âm, hai chuyện trùng nhau. Nhưng nếu bạn chặt trên một khoảng có biên âm, ví dụ tọa độ trong bài D nếu đề cho phép, thì $(-7) // 2$ bằng -4 trong Python và -3 trong C++, và một bản dịch từng dòng sẽ sai lệch một đơn vị ở đúng nửa dưới trục số. Mọi bài của buổi này chặt trên khoảng không âm, nên chuyện này chưa cần ai, nhưng nó là thứ phải biết trước khi gặp.

4. Hai mẫu chuẩn và bẫy làm tròn

Gần như mọi bài chặt nhị phân rơi vào một trong hai mẫu.

Tìm giá trị nhỏ nhất thoả mãn. Ví dụ: tốc độ nhỏ nhất, số phút nhỏ nhất.

```

while lo < hi:
    mid = (lo + hi) >> 1      # round DOWN
    if ok(mid):
        hi = mid
    else:
        lo = mid + 1

```

Tìm giá trị lớn nhất thoả mãn. Ví dụ: độ cao lưỡi cửa lớn nhất, khoảng cách nhỏ nhất lớn nhất có thể.

```

while lo < hi:
    mid = (lo + hi + 1) >> 1  # round UP
    if ok(mid):
        lo = mid
    else:
        hi = mid - 1

```

Chú ý dấu $+ 1$ ở mẫu thứ hai.

Vì sao bắt buộc phải làm tròn lên ở mẫu tìm giá trị lớn nhất. Khi $hi == lo + 1$, làm tròn xuống cho $mid == lo$, và nếu $ok(mid)$ đúng thì $lo = mid$ không đổi gì cả. Vòng lặp chạy mãi mãi.

Đây là cạm bẫy **sống sót nguyên vẹn** khi đổi ngôn ngữ, và ở Python nó còn khó thấy hơn một bậc. Chương trình không treo một cách ồn ào: nó chỉ quay vòng, hệ thống chấm trả về TLE, và **không có traceback** nào

cả vì chẳng có ngoại lệ nào được ném. Người làm bài nhìn thấy chữ TLE liền đi tối ưu hàm kiểm, trong khi thuật toán hoàn toàn đúng và chỗ hỏng nằm ở hai dòng cập nhật biên.

Cách nhớ gọn: **nhánh nào giữ điểm giữa lại thì phía đó phải được làm tròn về**. Mẫu một giữ mid ở hi nên làm tròn xuống, mẫu hai giữ mid ở lo nên làm tròn lên.

5. Chặt trên đáp án và điều kiện đơn điệu

Đây là phần chính của buổi.

Rất nhiều bài tối ưu có dạng: tìm giá trị tốt nhất của một đại lượng. Trả lời trực tiếp thì khó, nhưng câu hỏi **quyết định** thì dễ hơn hẳn: **với một giá trị X cho trước, có làm được không?**

Nếu hàm trả lời câu hỏi đó có tính **đơn điệu**, tức là làm được với X thì cũng làm được với mọi giá trị để hơn X, thì tập các X làm được là một nửa trục liên mạch, và ranh giới của nó tìm được bằng chặt nhị phân.

Ví dụ bài C: chia dãy thành k đoạn sao cho tổng lớn nhất của một đoạn là nhỏ nhất. Câu hỏi quyết định là: với giới hạn L, có chia được thành không quá k đoạn không? L càng lớn thì càng dễ, nên tính đơn điệu có sẵn.

Việc phải làm đầu tiên với mọi bài loại này là chứng minh tính đơn điệu, dù chỉ trong đầu và bằng một câu. Nếu không đơn điệu thì chặt nhị phân vẫn cho ra một con số, và con số đó sai một cách rất thuyết phục.

Độ phức tạp luôn có dạng: chi phí một lần kiểm nhân với logarit của khoảng giá trị. Với n bằng $2 \cdot 10^5$ và đáp án tới 10^{18} , đó là $2 \cdot 10^5$ nhân 60, khoảng 10^7 lượt lặp. Trong C++ đó là chuyện nhỏ. **Trong Python đó là biên giới**: mười triệu lượt lặp của trình thông dịch mất cỡ 1 tới 2 giây, nên ước lượng số lượt lặp trước khi gõ là việc bắt buộc chứ không phải việc tùy hứng.

Cách hạ con số đó xuống, theo thứ tự đáng thử:

- **Siết cận trên**. Bài B đặt cận trên $2 \cdot 10^9$ thay vì 10^{18} : 31 vòng thay vì 60, và đo được 233 ms thay vì 428 ms.
- **Cho hàm kiểm ít việc hơn**, ví dụ dựng sẵn mảng khoảng trống ở bài J thay vì tính hiệu trong mỗi vòng.
- **Bỏ mọi lời gọi hàm khỏi vòng trong**, xem mục 12.

6. Hàm kiểm thường là tham lam

Đây là quan sát đáng giá nhất của buổi, và là lý do chặt nhị phân hay được ghép chung với tham lam.

Trong hầu hết bài chặt trên đáp án, hàm kiểm cài bằng một lượt quét tham lam:

- Bài C: nhồi phần tử vào đoạn hiện tại cho tới khi vượt giới hạn, rồi mở đoạn mới. Cách này dùng ít đoạn nhất có thể với giới hạn đó.
- Bài D: luôn lấy vị trí đầu tiên, rồi lấy vị trí tiếp theo nào cách vị trí vừa lấy ít nhất d. Cách này đặt được nhiều bò nhất có thể với khoảng cách đó.
- Bài J: mỗi khoảng trống dài g cần đúng $\text{ceil}(g/d) - 1$ trạm chèn thêm, không hơn.

Điểm chung: tham lam ở đây **không cần chứng minh phức tạp**, vì nó tối ưu theo đúng đại lượng mà câu hỏi quyết định đang hỏi.

Và điểm chung thứ hai, quan trọng với người viết Python: hàm kiểm là một **vòng lặp chạy trên toàn bộ dữ liệu**, và nó chạy lại từ đầu trong mỗi vòng chặt. Đó là chỗ toàn bộ thời gian của bài nằm ở đó. Vòng chặt bên ngoài chỉ chạy vài chục lượt và không đáng kể.

7. Chọn cận trên cho đúng

Cận dưới thường hiển nhiên. Cận trên là chỗ hay sai, và ở Python nó chỉ còn sai theo **một** kiểu thay vì hai.

Đặt quá thấp thì mất nghiệm. Bài G hỏi số phút nhỏ nhất để k máy làm xong m sản phẩm. Đặt cận trên bằng m là phản xạ tự nhiên, vì m là con số lớn duy nhất nhìn thấy trong đề. Nhưng một máy chậm cần t_i

phút cho mỗi sản phẩm, nên đáp án có thể lên tới $t_{\min} \cdot m$, cỡ 10^{18} . Chặt trong một khoảng thiếu nghiêm trọng **không hề báo lỗi**: nó lặng lẽ trả về đúng cái cận trên bạn đưa vào, và con số đó trông vẫn hợp lý.

Đặt quá cao thì... trong C++ là tràn. Bài B tìm căn bậc hai nguyên của n tới 10^{18} , và bản C++ cảnh báo rằng đặt cận trên bằng n làm $\text{mid} * \text{mid}$ vượt long long.

Trong Python thì không. $\text{mid} * \text{mid}$ với mid cỡ $5 \cdot 10^{17}$ cho ra một số nguyên $2,5 \cdot 10^{35}$ hoàn toàn chính xác. Cận trên quá cao ở đây chỉ tốn thời gian, không làm sai kết quả:

Cận trên của bài B	Thời gian	Kết quả
$2 \cdot 10^9$	233 ms	đúng
n , tức tới 10^{18}	428 ms	vẫn đúng

Đây là một chỗ **bản C++ nói quá với Python**, và nó đáng nói ra vì một người đọc tài liệu C++ rồi dịch sang Python sẽ đi sửa một lỗi không tồn tại. Quy tắc còn lại vẫn nguyên giá trị, chỉ đổi lý do. Cận trên phải là một giá trị **chắc chắn thoả mãn**, và nên chọn giá trị nhỏ nhất mà bạn chứng minh được là thoả mãn. Trong C++ để tránh tràn, trong Python để tiết kiệm vòng lặp.

8. Chia lấy trần, và vì sao // của Python giúp bạn

Xuất hiện ở bài F và bài J. Ba cách viết, và chỉ hai cách đúng:

Cách viết	Đúng khi nào
<code>-(-a // b)</code>	mọi số nguyên, b khác 0
<code>(a + b - 1) // b</code>	a và b đều dương
<code>math.ceil(a / b)</code>	chỉ khi a và b nhỏ hơn 2^{53}

Cách thứ nhất đúng được là **nhờ Python làm tròn về phía âm vô cùng**. `-a // b` cho `-ceil(a/b)`, lấy dấu âm lần nữa là ra trần. Trong C++ công thức ấy sai, vì phép chia nguyên của C++ cắt về phía không. Đây là một chỗ hiểm hoai ngôn ngữ này tặng bạn một thứ ngôn ngữ kia không có.

Hai cách đầu đo ra như nhau, cùng 6 ms cho 200.000 phép chia, nên chọn cái nào cũng được.

Cách thứ ba thì không phải chuyện tốc độ. Số thực Python là double của C, đúng 53 bit đáng tin, trong khi 10^{18} cần 60 bit. Hai ví dụ chạy thật:

```
math.ceil(10**18 / 3) # 33333333333333312, tran dung la 33333333333333334
math.ceil(10**18 / 7) # 142857142857142864, tran dung la 142857142857142858
(10**400) / 3        # OverflowError: integer division result too large for a float
```

Lệch 22 đơn vị ở ví dụ thứ nhất, lệch 6 ở ví dụ thứ hai, và **lệch về hai chiều khác nhau**, nên không có cách nào bù lại. Ví dụ thứ ba đáng nhớ riêng: với số nguyên đủ lớn, a / b không ra số sai mà ném hẵn ngoại lệ, và kết luận của bộ chấm là RE chứ không phải WA.

Cùng lý do ấy, `math.sqrt` sai ở bài B, và `int(pow(n, 0.5))` cũng vậy.

9. Bốn cạm bẫy của bản C++ biến mất, và một cái mới xuất hiện

Bản C++ liệt kê tám cạm bẫy. Bốn cái trong đó là **tràn số**, và số nguyên Python không tràn, nên bốn cái đó không còn:

Cạm bẫy của bản C++	Trạng thái ở Python
$(lo + hi) / 2$ tràn khi hai biên cùng lớn	biến mất , xem mục 3
bài B đặt cận trên bằng n nên $mid \cdot mid$ tràn	biến mất , chỉ còn chậm hơn, xem mục 7
bài F để biến tổng số giờ là <code>int</code> nên cuộn về số bé	biến mất , xem chú thích trong <code>bai06.py</code>
bài G hàm kiểm tự tràn nên phải thoát sớm	biến mất , và xem mục 10
bài I biến đếm cấp để <code>int</code> , số cấp tới $2 \cdot 10^{10}$	biến mất

Bốn cạm bẫy còn lại **không đổi một chút nào**, vì chúng là lỗi thuật toán chứ không phải lỗi kiểu dữ liệu: làm tròn sai chiều, cận trên đặt hệt, lệch một dấu bằng trong hàm kiểm, và dùng số thực cho bài số nguyên.

Cái mới xuất hiện là thời gian. Trong C++ một hàm kiểm $O(n)$ chạy 60 lần trên n bằng $2 \cdot 10^5$ là $1,2 \cdot 10^7$ phép, không ai phải nghĩ. Trong Python đó là $1,2 \cdot 10^7$ lượt của trình thông dịch, cỡ 1,5 tới 2,5 giây, tức là vừa đúng ở ranh giới. Nên buổi này trong Python có một câu hỏi mà bản C++ không cần hỏi: **hàm kiểm của tôi chạy bao nhiêu lượt, và tôi có thoát được cận trên không.**

Nói cho đủ: bộ test kèm theo của buổi này **không** đẩy bài nào tới ranh giới ấy. Bài chậm nhất trong mười bài là bài I với 492 ms, và giới hạn của chủ đề là 2.000 ms. Con số $1,2 \cdot 10^7$ ở trên là ước lượng cho trường hợp xấu nhất mà **giới hạn của đề cho phép**, không phải số đo trên bộ test có thật.

10. Thoát sớm trong hàm kiểm: tám trên tám đều chậm đi

Bản C++ của bài G và bài J cho hàm kiểm **thoát sớm**: dừng cộng dồn ngay khi tổng đã đủ. Lý do ở đó rất cụ thể, và nó là lý do **đúng**: nếu cứ cộng tiếp thì tổng vượt 64 bit và hàm kiểm nói dối.

Ở Python lý do đó không còn. Vậy có nên giữ lối viết ấy như một khoản bảo hiểm miễn phí không? Câu hỏi đó đo được, nên tôi đo. Tám bài của buổi này có hàm kiểm dạng cộng dồn hoặc đếm. Với mỗi bài, chạy test lớn nhất hai lần, một lần có dòng thoát sớm và một lần không:

Bài	Có thoát sớm	Không thoát sớm
C	306 ms	292 ms
D	135 ms	133 ms
E	172 ms	162 ms
F	208 ms	171 ms
G	321 ms	282 ms
H	70 ms	61 ms
I	527 ms	506 ms
J	150 ms	139 ms

Tám trên tám đều chậm đi, từ 1,5 phần trăm tới 22 phần trăm, và kết quả in ra giống hệt nhau ở cả tám. Lý do dễ thấy khi đã có bảng: dòng `if s >= m: break` chạy trên **mọi** phần tử của **mọi** vòng chặt, còn nó chỉ cắt ngắn được ở một số ít vòng. Trong C++ khoản phí ấy là một phép so sánh máy, gần như không đo được; trong Python nó là một lệnh bytecode đầy đủ, ngang giá với chính phép cộng mà nó đang bảo vệ.

Nên mười lời giải mẫu của buổi này **không có dòng thoát sớm nào**. Đó không phải lười, đó là số đo.

Bài học rộng hơn, và nó đáng nhớ hơn cả bảng trên: **một tối ưu chép từ ngôn ngữ khác sang là một giả thiết, không phải một cải tiến.** Ở chủ đề 06 lời khuyên "gỡ hàm con thành mã thẳng" đo được 2,9 s xuống 2,7 s và nó đúng. Ở đây tôi thử lại đúng lời khuyên ấy cho hàm kiểm của bài C: **291 ms khi gỡ thẳng, 289 ms khi để trong hàm con.** Không có khác biệt, vì hàm kiểm chỉ được gọi 48 lần chứ không phải vài triệu lần như một nút của segment tree. Cùng một lời khuyên, cùng một ngôn ngữ, hai kết luận ngược nhau, và cái quyết định là **số lần gọi**, không phải cái tên của kỹ thuật.

11. Chặt nhị phân trên số thực

Bộ đề của buổi này không có bài số thực, vì bộ chấm so kết quả theo từng từ nên không kiểm được sai số. Nhưng đề thi thật có, nên mẫu vẫn cần nhớ:

```
lo = 0.0; hi = 1e9
for _ in range(100):           # fixed number of rounds, not an epsilon
    mid = (lo + hi) / 2
    if ok(mid):
        hi = mid
    else:
        lo = mid
print("%.9f" % lo)
```

Lặp số vòng cố định thay vì so với một epsilon. Vòng while $hi - lo > 1e-9$ có thể không bao giờ dừng khi hai số quá gần nhau tới mức phép trừ mất hết chữ số có nghĩa. Một trăm vòng chia đôi thu khoảng cách xuống còn 2^{-100} lần khoảng ban đầu, thừa chính xác cho mọi đề, và luôn dừng.

Chú ý ở dòng cuối: `print(x)` của Python in số thực theo repr, tức là số chữ số ngắn nhất dừng lại đúng giá trị, chứ không theo số chữ số đề yêu cầu. Đề đòi chín chữ số sau dấu chấm thì phải nói ra, bằng `("%.9f" % x hoặc f"{x:.9f}")`.

12. Cạm bẫy Python ngoài phần thuật toán

Cạm bẫy	Hậu quả
<code>input()</code> trong vòng lặp truy vấn	mỗi lời gọi là một lượt vào ra có khoá; với $2 \cdot 10^5$ dòng nó một mình ăn hết giới hạn
<code>print(x)</code> từng dòng / thay //	gom vào list rồi <code>'\{ }n'.join</code> và ghi một lần / luôn cho số thực, và số thực mất chính xác từ khoảng $9 \cdot 10^{15}$
<code>math.ceil(a / b)</code> cho số lớn	xem mục 8, lệch tới 22 đơn vị ở 10^{18}
gọi <code>min(a, b)</code> trong vòng nóng	lời gọi hàm thật; viết <code>a if a < b else b</code>
<code>sorted(...)</code> rồi quên gán	<code>sorted</code> trả về mảng mới, <code>list.sort()</code> mới là sửa tại chỗ
chặt trên mảng chưa sắp	<code>bisect</code> không kiểm và không báo, nó chỉ trả một con số sai

Dòng đầu là dòng đáng tiền nhất. Mọi lời giải của buổi này mở đầu bằng:

```
data = sys.stdin.buffer.read().split()
```

Đọc hết đầu vào thành một khối byte rồi tách một lần, sau đó đi bằng chỉ số. Test lớn nhất của bài A là 4 MB và 400.000 số; đọc kiểu này mất một phần nhỏ của 264 ms tổng, còn `input()` cho mỗi dòng thì riêng nó đã vượt giới hạn.

13. Bộ khung tự hỏi khi gặp đề mới

Thấy đề hỏi giá trị nhỏ nhất của một cái lớn nhất, hoặc lớn nhất của một cái nhỏ nhất, gần như chắc chắn là chặt trên đáp án. Khi đó hỏi tiếp:

- Có mảng đã sắp để chặt không? Có thì `bisect_left` hoặc `bisect_right` đã xong việc, và phải chọn đúng một trong hai.
- Không có mảng thì câu hỏi quyết định là gì? Viết ra thành một câu tiếng Việt **trước khi** viết code.
- Nó có đơn điệu không? Nếu X làm được thì X dễ hơn có làm được không?

- Đang tìm giá trị nhỏ nhất hay lớn nhất thoả mãn? Chọn đúng một trong hai mẫu ở mục 4, và kiểm lại dấu + 1 của điểm giữa.
- Cận dưới và cận trên là gì? Cận trên có **chắc chắn** thoả mãn không, và có siết nhỏ lại được không?
- Hàm kiểm chạy bao nhiêu lượt lặp Python? Nhân với số vòng chặt. Quá 10^7 thì phải nghĩ lại trước khi gõ.
- Có phép chia nào trong hàm kiểm không? Nếu có thì nó là chia lấy sàn hay lấy trần, và bạn đã viết đúng chưa?

Kế tiếp: sang **Phần II** làm bài. Làm xong hãy tự chấm bằng `chay.py` trước khi mở **Phần III**.

Phần II. Mười bài tập

Đề PDF	Ở đây	Kỹ thuật chính	File nộp
A	Bài 1	viết tay vòng chặt, cấm bisect	bai01.py
B	Bài 2	chặt thay cho hàm căn	bai02.py
C	Bài 3	chặt trên đáp án, kiểm bằng tham lam	bai03.py
D	Bài 4	mẫu tìm giá trị lớn nhất	bai04.py
E	Bài 5	mẫu tìm giá trị lớn nhất	bai05.py
F	Bài 6	chặt kèm chia lấy trần	bai06.py
G	Bài 7	chọn đúng cận trên	bai07.py
H	Bài 8	chặt trên giá trị, kiểm bằng đếm	bai08.py
I	Bài 9	chặt kèm hai con trỏ	bai09.py
J	Bài 10	chặt kèm chia lấy trần	bai10.py

Giới hạn thời gian **2 giây** cho mọi bài, gấp đôi bản C++ vì đây là nhánh Python. Con số 2 giây đặt theo số đo chứ không theo một tỉ lệ: bài chậm nhất trong mười bản mẫu là bài I với 492 ms, tức 25 phần trăm giới hạn. Chạy lại nhiều lượt thì con số này dao động trong khoảng 490 tới 560 ms, nên giới hạn để rộng gấp bốn lần. Tự chấm:

```
python3 chay.py topics/03-binary-search/solutions/bai01.py
python3 chay.py topics/03-binary-search --tat-ca
```

Bài A. Đếm phần tử nhỏ hơn

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên đã sắp xếp không giảm và q truy vấn. Mỗi truy vấn cho một số x , hãy đếm số phần tử của dãy có giá trị **nhỏ hơn thật sự** x .

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên không giảm a_i ($|a_i| \leq 10^9$)
- q dòng tiếp theo, mỗi dòng một số x ($|x| \leq 10^9$)

Kết quả

q dòng, mỗi dòng một đáp án.

Ví dụ

Dữ liệu vào	Kết quả
5 4	0
1 3 3 5 7	1
0	3
3	5
4	
100	

Lưu ý

Bài này **cấm dùng bisect**, cả `bisect_left` lẫn `bisect_right`. Mục đích là để bạn viết tay cho đúng biên, vì chín bài sau không có mảng nào để gọi thư viện. Và khi viết tay, hãy để ý đúng một dấu: `<` cho ra `bisect_left`, `<=` cho ra `bisect_right`, và đề này hỏi cái thứ nhất.

Giải thích

Dãy đã sắp là 1, 3, 3, 5, 7. Không phần tử nào nhỏ hơn 0. Chỉ số 1 nhỏ hơn 3. Ba phần tử 1, 3, 3 nhỏ hơn 4. Cả năm phần tử đều nhỏ hơn 100.

Bài B. Căn bậc hai nguyên

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho q số nguyên không âm. Với mỗi số n , hãy tính phần nguyên của căn bậc hai của n , tức là số nguyên r lớn nhất thỏa mãn $r \cdot r \leq n$.

Dữ liệu vào

- Dòng 1: q ($1 \leq q \leq 10^5$)
- q dòng tiếp theo, mỗi dòng một số n ($0 \leq n \leq 10^{18}$)

Kết quả

q dòng, mỗi dòng một đáp án.

Ví dụ

Dữ liệu vào	Kết quả
5	0
0	1
1	1
2	3
15	4
16	

Lưu ý

Bài này cấm cả `math.sqrt` lẫn `math.isqrt` và `pow(n, 0.5)`. Cái thứ nhất và thứ ba sai thật: số thực chỉ giữ được 53 bit đáng tin, trong khi 10^{18} cần 60 bit, nên kết quả lệch một đơn vị ở đúng những chỗ khó thấy nhất. Cái thứ hai, `math.isqrt`, thì **đúng tuyệt đối và nhanh hơn hẳn**, nên cấm nó là cấm cho mục đích luyện tập chứ không phải vì nó hỏng. Đây là mẫu tìm giá trị **lớn nhất**, hãy soi kỹ dấu $+ 1$ của điểm giữa.

Giải thích

Căn bậc hai nguyên là số nguyên lớn nhất mà bình phương không vượt quá đầu vào. Với 2 thì đó là 1 vì $1^2 = 1 \leq 2$ còn $2^2 = 4 > 2$. Với 15 thì đó là 3, còn 16 là số chính phương nên được đúng 4.

Bài C. Chia dãy thành k đoạn

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên dương. Hãy chia dãy thành đúng k đoạn liên tiếp không rỗng sao cho **tổng lớn nhất trong các đoạn là nhỏ nhất có thể**. Hãy in giá trị nhỏ nhất đó.

Dữ liệu vào

- Dòng 1: n và k ($1 \leq k \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
5 3 1 2 3 4 5	6

Giải thích

Chia thành 1 2 3, 4, 5 cho tổng lớn nhất là 6. Mọi cách chia khác đều có một đoạn nặng hơn.

Lưu ý

Đừng đi tìm cách chia. Hãy hỏi câu dễ hơn: với một giới hạn cho trước, có chia được thành không quá k đoạn không? Và khi viết câu điều kiện của lượt quét tham lam, hãy tự hỏi dấu so sánh có được phép mang dấu bằng không.

Bài D. Đặt bò vào chuồng

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Có n chuồng đặt tại các vị trí $x_1 \dots x_n$ trên một đường thẳng, và k con bò. Hai con bò đứng quá gần nhau sẽ đánh nhau, nên bạn muốn đặt k con bò vào k chuồng khác nhau sao cho **khoảng cách nhỏ nhất giữa hai con bò bất kỳ là lớn nhất có thể**. Hãy in khoảng cách đó.

Dữ liệu vào

- Dòng 1: n và k ($2 \leq k \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên đôi một khác nhau x_i ($|x_i| \leq 10^{18}$), không nhất thiết đã sắp xếp

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
5 3 1 2 8 4 9	3

Giải thích

Đặt bò ở các vị trí 1, 4 và 8 thì hai khoảng cách là 3 và 4, nhỏ nhất là 3. Không có cách nào đạt 4.

Lưu ý

Bài này tìm giá trị *lớn nhất* thoả mãn. Hãy kiểm tra thật kỹ cách bạn tính điểm giữa, vì hai mẫu làm tròn theo hai chiều ngược nhau, và chọn nhầm ở đây không cho WA mà cho **TLE không kèm traceback**: chương trình chỉ quay vòng, không ném ngoại lệ nào, nên chẳng có dấu vết nào chỉ vào hai dòng cập nhật biên.

Bài E. Cắt gỗ

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Có n cây gỗ, cây thứ i cao a_i . Bạn đặt lưỡi cưa ở độ cao h và cắt ngang, thu được phần trên của mọi cây cao hơn h , tức là tổng của $a_i - h$ trên các cây có $a_i > h$.

Hãy tìm độ cao h **lớn nhất** là số nguyên không âm sao cho lượng gỗ thu được vẫn ít nhất M .

Dữ liệu vào

- Dòng 1: n và M ($1 \leq n \leq 2 \cdot 10^5, 1 \leq M \leq 2 \cdot 10^{14}$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^9$)

Đề bảo đảm tổng của toàn bộ a_i không nhỏ hơn M , nên luôn có đáp án.

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
4 7 20 15 10 17	15

Giải thích

Với $h = 15$ thu được $5 + 0 + 0 + 2 = 7$ mét. Với $h = 16$ chỉ còn 5 mét.

Lưu ý

Cạm bẫy của bài này nằm trong hàm tính lượng gỗ chứ không nằm ở vòng chặt. Bản C++ còn một cảnh báo nữa là tổng lên tới $2 \cdot 10^{14}$ nên phải để `long long`; ở Python cảnh báo ấy không áp dụng, số nguyên tự lớn lên.

Bài F. Tốc độ ăn chuỗi

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Có n đồng chuỗi, đồng thứ i có a_i quả. Bạn chọn một tốc độ ăn v là số nguyên dương. Mỗi giờ bạn chọn một đồng và ăn tối đa v quả từ đồng đó, nếu đồng còn ít hơn v quả thì ăn hết và giờ đó vẫn tính là một giờ trọn vẹn.

Hãy tìm tốc độ v **nhỏ nhất** sao cho ăn hết toàn bộ chuỗi trong không quá h giờ.

Dữ liệu vào

- Dòng 1: n và h ($1 \leq n \leq 2 \cdot 10^5$, $n \leq h \leq 10^{18}$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
4 8 3 6 7 11	4

Giải thích

Với $v = 4$ số giờ là $1 + 2 + 2 + 3 = 8$. Với $v = 3$ là $1 + 2 + 3 + 4 = 10$, quá 8 giờ.

Lưu ý

Số giờ cho một đồng là phần nguyên **trên** của một phép chia. Hãy viết nó bằng số nguyên, $-(-a // v)$ hoặc $(a + v - 1) // v$, đừng mượn `math.ceil` của số thực. Bản C++ còn cảnh báo rằng tổng số giờ đạt $2 \cdot 10^{14}$ nên biến tổng phải đủ rộng; ở Python chuyện đó không tồn tại, nhưng chính vì vậy mà **bộ test 03 dựng riêng để bắt lỗi tràn của bản C++ lại không kiểm được gì ở đây**, và lỗi duy nhất còn sống là chia nhầm sà thay vì trần.

Bài G. Máy sản xuất

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Có k máy, máy thứ i cần t_i phút để làm xong một sản phẩm. Các máy chạy song song và độc lập. Hãy tìm số phút **nhỏ nhất** để cả xưởng làm được ít nhất m sản phẩm.

Dữ liệu vào

- Dòng 1: k và m ($1 \leq k \leq 2 \cdot 10^5, 1 \leq m \leq 10^9$)
- Dòng 2: k số nguyên t_i ($1 \leq t_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
3 7 3 2 5	8

Giải thích

Trong 8 phút, ba máy làm được lần lượt 2, 4 và 1 sản phẩm, tổng là 7. Trong 7 phút chỉ được 6 sản phẩm.

Lưu ý

Hãy tính thử đáp án lớn nhất có thể xảy ra **trước khi** chọn cận trên. Đặt hệt cận trên không gây lỗi nào nhìn thấy được: vòng chặt trả về đúng cái cận bạn đưa vào, và con số ấy trông hợp lý. Bản C++ còn dặn cho hàm kiểm thoát sớm để tổng khỏi tràn 64 bit; ở Python thì không cần, và mục I.10 đo ra rằng thêm dòng thoát sớm còn làm bài này **chậm đi** từ 282 ms lên 321 ms.

Bài H. Phần tử thứ k của bảng nhân

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Xét bảng nhân kích thước n dòng và m cột, trong đó ô ở dòng i cột j chứa giá trị $i \cdot j$. Xếp toàn bộ $n \cdot m$ giá trị theo thứ tự không giảm, hãy cho biết giá trị thứ k .

Dữ liệu vào

- Một dòng chứa n , m và k ($1 \leq n, m \leq 5 \cdot 10^4, 1 \leq k \leq n \cdot m$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
3 3 5	3

Giải thích

Bảng gồm các giá trị 1, 2, 3, 2, 4, 6, 3, 6, 9. Sắp lại thành 1, 2, 2, 3, 3, 4, 6, 6, 9, và phần tử thứ 5 là 3.

Lưu ý

$n \cdot m$ có thể đạt $2,5 \cdot 10^9$ nên bảng không dựng ra được. Khi đếm số ô không vượt một giá trị, hãy nhớ mỗi dòng chỉ có đúng m ô. Với dữ liệu mẫu ba nhân ba thì quên điều đó vẫn ra đúng, nên lỗi sống sót tới test lớn.

Bài I. Khoảng cách cặp nhỏ thứ k

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho n điểm trên trục số. Xét toàn bộ $n(n-1)/2$ cặp điểm và khoảng cách của chúng, xếp các khoảng cách này theo thứ tự không giảm. Hãy cho biết khoảng cách thứ k .

Dữ liệu vào

- Dòng 1: n và k ($2 \leq n \leq 2 \cdot 10^5$, $1 \leq k \leq n(n-1)/2$)
- Dòng 2: n số nguyên x_i ($|x_i| \leq 10^9$), có thể trùng nhau

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
4 3 1 3 6 10	4

Giải thích

Sáu khoảng cách là 2, 5, 9, 3, 7, 4. Sắp lại thành 2, 3, 4, 5, 7, 9 nên phần tử thứ 3 là 4.

Lưu ý

k có thể đạt gần $2 \cdot 10^{10}$ nên không liệt kê cặp được. Đây là bài duy nhất của buổi có sẵn một mảng đã sắp để bisect làm việc, và cũng là bài mà bisect **thua** lượt quét hai con trỏ viết bằng Python thuần; mục I.2 có số đo. Khi đếm, hãy tự hỏi một cặp gồm một điểm với chính nó có được tính không.

Bài J. Thêm trạm xăng

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Trên một quốc lộ thẳng có n trạm xăng tại các vị trí $x_1 \dots x_n$. Bạn được xây thêm tối đa k trạm nữa, mỗi trạm ở một vị trí nguyên bất kỳ trên quốc lộ.

Hãy làm cho **khoảng cách lớn nhất giữa hai trạm liên tiếp là nhỏ nhất có thể**, và in giá trị đó.

Dữ liệu vào

- Dòng 1: n và k ($2 \leq n \leq 2 \cdot 10^5$, $0 \leq k \leq 10^9$)
- Dòng 2: n số nguyên đôi một khác nhau x_i ($|x_i| \leq 10^9$), không nhất thiết đã sắp xếp

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
4 3 1 5 9 13	2

Giải thích

Ba khoảng trống đều dài 4. Xây thêm một trạm vào giữa mỗi khoảng, vừa đúng 3 trạm, thì mọi khoảng cách còn 2.

Lưu ý

Một khoảng trống dài g cần thêm bao nhiêu trạm để không đoạn nào dài quá d ? Trả lời được câu đó là xong bài. Và khi trả lời, hãy để ý rằng viết nhầm chia lấy sàn thành chia lấy trần ở đây sai **theo hướng dễ tin nhất**: nó báo cần ít trạm hơn thực tế, nên đáp án in ra nhỏ hơn đáp án đúng, mà nhỏ hơn thì trông đẹp hơn.

Phần III. Gợi ý và lời giải

Đọc từng tầng một, dừng lại ngay khi đã nghĩ ra. Code đầy đủ nằm ở Phần IV.

Bài	Cái bẫy thật sự	Riêng với Python
A	cập nhật sai biên nên đánh mất chính đáp án	bisect_left nhanh gấp 2,4 lần, nhưng dễ nhầm
B	mẫu lớn nhất mà làm tròn xuống, vòng lặp không thoát	mid * mid không tràn, cận trên chỉ ảnh hưởng tốc độ
C	hàm kiểm tra lam lệch một dấu bằng	gỡ hàm con ra không đo được khác biệt
D	tìm giá trị lớn nhất mà làm tròn xuống	TLE không kèm traceback, không có gì chỉ đường
E	quên bỏ qua cây thấp hơn lưỡi cưa	cảnh báo long long của bản C++ không áp dụng
F	chia lấy sàn thay vì lấy trần	- (-a // v) đúng nhờ // làm tròn về âm vô cùng
G	cận trên đặt bằng m, thiếu hẳn vùng nghiệm	thoát sớm làm chậm đi 282 ms lên 321 ms
H	quên chặn số ô của một dòng lại ở mức m	min trong vòng nóng là một lời gọi hàm thật
I	hai con trỏ đếm dư mất một cặp	bisect thua hai con trỏ, 1.143 ms so với 523 ms
J	chia lấy sàn thay vì lấy trần	như bài F

Bài A. Đếm phần tử nhỏ hơn

Gợi ý 1. Dãy đã sắp xếp. Nếu $a[mid] < x$ thì mọi phần tử từ đầu tới mid đều nhỏ hơn x , vậy đáp án ít nhất là $mid + 1$.

Gợi ý 2. Nếu $a[mid] \geq x$ thì mid không được tính, nhưng những vị trí **trước** nó vẫn có thể được tính. Chú ý chữ "vẫn có thể": nghĩa là chưa được phép loại mid khỏi cửa sổ theo kiểu $hi = mid - 1$.

Gợi ý 3. Bất biến nửa mở: đáp án luôn nằm trong $[lo, hi]$, và vòng lặp dừng khi $lo == hi$.

Lời giải.

```
lo = 0; hi = n
while lo < hi:
    mid = (lo + hi) >> 1
    if a[mid] < x:
        lo = mid + 1
    else:
        hi = mid
```

Ba chỗ quyết định tính đúng đắn:

- **Hai nhánh không đối xứng.** Nhánh loại bỏ dùng $mid + 1$, nhánh giữ lại dùng mid. Viết $hi = mid - 1$ ở nhánh thứ hai thì khi $a[mid]$ đúng bằng x , vị trí mid chính là đáp án và bạn vừa vứt nó đi. Lỗi này không lộ trên dữ liệu mẫu: ví dụ của đề vẫn ra đủ bốn dòng đúng, và chỉ test 200.000 truy vấn mới bắt được.
- **Dấu so sánh là nhỏ hơn thật sự.** Viết $<$, không phải $\leq$. Đổi thành $\leq$ là bạn vừa cài bisect_right thay cho bisect_left, tức là trả lời một câu hỏi khác. Với ví dụ của đề, truy vấn $x = 3$ sẽ ra 3 thay vì 1.

- **Điểm giữa không cần công thức chống tràn.** Viết $(lo + hi) \gg 1$ là đủ ở Python, không cần $lo + (hi - lo) // 2$, và lỗi viết đó còn chậm hơn 20 phần trăm; xem bảng ở mục I.3.

Python. `bisect_left(a, x)` trả về đúng con số này và nhanh hơn 2,4 lần, 109 ms so với 264 ms trên test lớn nhất. Đề cấm nó, nhưng trong kỳ thi thật, nếu bài chỉ cần chặt trên một mảng có sẵn thì gọi thư viện là lựa chọn đúng, cả về thời gian chạy lẫn về số lỗi biên bạn có thể mắc.

Bài B. Căn bậc hai nguyên

Gợi ý 1. Bạn đang tìm số r lớn nhất thoả mãn $r \cdot r \leq n$. Điều kiện này có đơn điệu theo r không?

Gợi ý 2. Có. r càng lớn thì $r \cdot r$ càng lớn, nên tập các r thoả mãn là một đoạn liên từ 0 trở lên. Đây là mẫu tìm giá trị lớn nhất, phải làm tròn lên.

Gợi ý 3. Cận trên nên đặt bằng bao nhiêu? Đáp án lớn nhất có thể là bao nhiêu khi n bằng 10^{18} ?

Lời giải. Đáp án không bao giờ vượt 10^9 , nên đặt cận trên bằng $2 \cdot 10^9$ là đủ.

```
lo = 0; hi = 2000000000
while lo < hi:
    mid = (lo + hi + 1) >> 1
    if mid * mid <= n:
        lo = mid
    else:
        hi = mid - 1
```

Dấu cộng một ở điểm giữa là cả bài. Bỏ nó đi thì khi $hi == lo + 1$ ta có $mid == lo$, và nếu $mid * mid \leq n$ thì $lo = mid$ không đổi gì cả. Vòng lặp chạy mãi. Bộ chấm trả về TLE trong khi thuật toán đúng hoàn toàn.

Dấu bằng trong điều kiện cũng là cả bài. Viết $< n$ thay cho $\leq n$ thì với mọi số chính phương bạn trả về đáp án nhỏ hơn một đơn vị: với $n = 1$ kết quả thành 0, với $n = 16$ thành 3. Ví dụ ngay trong đề bắt được lỗi này.

Hai cái bẫy của bản C++, và chỉ một cái còn sống.

- **Dùng hàm căn.** Vẫn còn sống. `math.sqrt` trả về double với 53 bit đáng tin, còn 10^{18} cần 60 bit, nên với số chính phương lớn hoặc số ngay dưới nó, kết quả lệch một đơn vị. Test cỡ tối đa cố tình nhồi rất nhiều giá trị dạng $r \cdot r - 1$ chính vì vậy. `pow(n, 0.5)` cũng là số thực, cùng một lỗi.
- **Đặt cận trên bằng n nên $mid \cdot mid$ tràn. Không còn.** Ở Python $mid * mid$ với mid cỡ $5 \cdot 10^{17}$ cho một số nguyên $2,5 \cdot 10^{35}$ hoàn toàn chính xác. Cận trên rộng chỉ làm chậm: 428 ms thay vì 233 ms, vì số vòng tăng từ 31 lên 60 và mỗi phép nhân phải làm trên số nhiều từ máy. Kết quả vẫn đúng từng dòng.

Python. `math.isqrt(n)` có từ 3.8, chính xác tuyệt đối với mọi số nguyên, và đo được 26 ms cho cả bài, tức nhanh hơn vòng tự viết chín lần. Trong kỳ thi thật thì đó là câu trả lời. Ở đây nó bị cấm vì buổi học cần bạn viết được vòng lặp.

Bài C. Chia dãy thành k đoạn

Gợi ý 1. Đừng tìm cách chia. Hãy hỏi câu dễ hơn: với giới hạn L cho trước, có chia được thành không quá k đoạn mà mỗi đoạn có tổng không vượt L không?

Gợi ý 2. Câu hỏi đó đơn điệu: L càng lớn càng dễ. Vậy chặt nhị phân trên L .

Gợi ý 3. Với L cố định, cách chia dùng ít đoạn nhất là tham lam: nhồi vào đoạn hiện tại tới khi vượt L thì mở đoạn mới.

Lời giải. Cận dưới là phần tử lớn nhất, vì một đoạn phải chứa trọn nó. Cận trên là tổng cả dãy, vì một đoạn duy nhất luôn hợp lệ. Độ phức tạp $O(n \log S)$, với S là tổng cả dãy.

```

cnt = 1; cur = 0
for v in a:
    if cur + v > mid:
        cnt += 1; cur = v
    else:
        cur += v

```

Điều kiện phải là dấu lớn hơn thật sự. Viết $cur + v \geq mid$ thì bạn cắt đoạn sớm hơn một bước, dùng nhiều đoạn hơn mức cần, và đáp án bị đẩy lên cao hơn thực tế. Đây là kiểu lỗi lệch một dấu bằng mà đọc code rất khó thấy, vì hai dòng chỉ khác nhau một ký tự và cả hai đều "trông hợp lý".

Chú ý hàm kiểm dùng $\leq k$ chứ không phải $== k$. Chia được thành ít đoạn hơn k thì luôn tách thêm được cho đủ, vì mọi phần tử đều dương.

Python, và đây là chỗ tôi đã đoán sai rồi đo lại. Chủ đề 06 dạy rằng phải gỡ hàm con thành mã thẳng, vì mỗi lời gọi hàm là thời gian thật. Tôi chép lời khuyên ấy sang đây, rồi đo: **291 ms khi viết thẳng, 289 ms khi để hàm kiểm trong một hàm con.** Không có khác biệt. Lý do là hàm kiểm ở đây chỉ được gọi 48 lần, mỗi lần làm $2 \cdot 10^5$ việc; ở segment tree thì hàm con được gọi vài triệu lần, mỗi lần làm một việc. Cái quyết định là **số lần gọi**, không phải cái tên của kỹ thuật. Lời giải mẫu vẫn viết thẳng, nhưng vì lý do trình bày chứ không phải vì tốc độ.

Bài D. Đặt bò vào chuồng

Gợi ý 1. Câu hỏi quyết định: với khoảng cách d , có đặt được ít nhất k con bò không?

Gợi ý 2. Đơn điệu theo chiều ngược với bài C: d càng nhỏ càng dễ. Vậy đây là mẫu tìm giá trị **lớn nhất** thoả mãn.

Gợi ý 3. Với d cố định, cách đặt được nhiều bò nhất là tham lam: luôn lấy chuồng đầu tiên, rồi lấy chuồng tiếp theo nào cách chuồng vừa lấy ít nhất d .

Lời giải. Sắp xếp trước, rồi chắt trên d trong khoảng từ 0 tới khoảng cách hai đầu.

```

mid = (lo + hi + 1) >> 1      # round UP, this is the max pattern
if cnt >= k:
    lo = mid
else:
    hi = mid - 1

```

Đây là bài để bạn nhớ bẫy làm tròn. Viết $(lo + hi) >> 1$ thì khi $hi == lo + 1$ ta có $mid == lo$, và nếu $ok(mid)$ đúng thì $lo = mid$ không đổi, vòng lặp chạy mãi.

Ở Python chuyện này khó lần ra hơn ở C++ đúng một bậc, và lý do rất cụ thể: người mới thường nghĩ rằng chương trình Python sai thì sẽ có traceback. Ở đây **không có gì cả**. Không ngoại lệ, không cảnh báo, không dấu hiệu. Chỉ có một tiến trình quay vòng cho tới khi bộ chấm giết nó, và một chữ TLE. Chữ TLE đẩy người làm bài đi tối ưu hàm kiểm, tức là đi xa khỏi chỗ hỏng.

Vì sao tham lam đúng: đặt con bò đầu tiên ở chuồng ngoài cùng bên trái không bao giờ thiệt, vì mọi lời giải khác đều có thể dịch con bò trái nhất về đó mà không làm hỏng ràng buộc. Lập luận này gọi là đối số đổi chỗ.

Python. `sorted(map(int, data[2:2 + n]))` làm cả việc ép kiểu lẫn việc sắp xếp trong C, không cần một vòng lặp Python nào. Và cảnh báo `long long` của bản C++ cho toạ độ tới 10^{18} thì không áp dụng.

Bài E. Cắt gỗ

Gợi ý 1. Lượng gỗ thu được giảm dần khi h tăng. Vậy tập các h hợp lệ là một đoạn liên từ 0 lên tới đáp án.

Gợi ý 2. Mẫu tìm giá trị lớn nhất, làm tròn lên.

Gợi ý 3. Cận trên là cây cao nhất, vì đặt lưỡi cao hơn thế thì không thu được gì.

Lời giải. Độ phức tạp $O(n \log H)$, với H là chiều cao của cây cao nhất.

Cái bẫy nằm ở hàm tính lượng gỗ, không phải ở vòng chặt:

```
for v in a:
    if v > mid:           # the guard is the whole point of this problem
        s += v - mid
```

Bỏ câu điều kiện thì những cây thấp hơn lưỡi cưa đóng góp một lượng **âm**, và tổng bị kéo xuống. Với ví dụ của đề, tổng thành $62 - 4h$ và đáp án ra 13 thay vì 15.

Một cảnh báo của bản C++ không áp dụng ở đây. Bản C++ dẫn biến s phải là long long vì tổng lên tới $2 \cdot 10^{14}$. Số nguyên Python tự lớn lên, nên câu đó không còn nghĩa gì. Nếu bạn đọc tài liệu C++ rồi dịch sang, đừng mất thời gian đi tìm chỗ khai báo kiểu.

Python. Có thể viết hàm kiểm thành `sum(v - mid for v in a if v > mid)`, ngắn hơn và chạy tương đương. Lời giải mẫu giữ vòng for tường minh để dòng chứa cái bẫy nhìn thấy rõ.

Bài F. Tốc độ ăn chuỗi

Gợi ý 1. Với tốc độ v , đồng có a quả tốn bao nhiêu giờ?

Gợi ý 2. Tồn phần nguyên **trên** của a / v , vì giờ cuối cùng vẫn tính trọn dù ăn chưa đầy.

Gợi ý 3. Tổng số giờ giảm khi v tăng, nên đây là mẫu tìm giá trị nhỏ nhất.

Lời giải.

```
s = 0
for v in a:
    s += -(v // mid)    # ceiling division, never a plain //
```

Ba chỗ mất điểm, và ở Python chỉ còn hai.

- **Chia lấy sàn thay vì lấy trần.** Tức là viết `v // mid`. Còn sống, và là cái bẫy chính của bài. Số giờ bị đếm thiếu, tốc độ trả về nhỏ hơn thực tế. Ví dụ của đề bắt được ngay: đáp án thành 3 thay vì 4.
- **Mượn hàm làm tròn của số thực**, tức `math.ceil(v / mid)`. Còn sống, xem mục I.8. Với v cỡ 10^9 thì thường vẫn đúng, nhưng đây là thói quen sẽ giết bạn ở bài có giá trị cỡ 10^{18} , và h của bài này đã được phép lên tới 10^{18} .
- **Biến tổng để kiểu int. Không còn.** Bản C++ ghi rõ rằng test 03 được dựng riêng để bắt lỗi này: 200.000 đồng, mỗi đồng 21.475 quả, nên ở tốc độ 1 tổng số giờ là 4.295.000.000, vượt 2^{32} và cuộn về một số bé xíu. Ở Python không có gì cuộn cả, nên **test 03 vẫn chạy nhưng không còn kiểm được điều nó sinh ra để kiểm**. Nó vẫn có ích, chỉ là với tư cách một test lớn bình thường.

Vì sao công thức đổi dấu hai lần lại đúng. `//` của Python làm tròn về phía âm vô cùng, nên `-a // b` cho `-ceil(a/b)`, và đổi dấu lần nữa là ra trần. Công thức này đúng cho mọi số nguyên. Công thức `(a + b - 1) // b` chỉ đúng khi cả hai dương, nhưng ở bài này cả hai đều dương và hai cách đo ra như nhau.

Bài G. Máy sản xuất

Gợi ý 1. Trong T phút, máy i làm được bao nhiêu sản phẩm?

Gợi ý 2. Đúng bằng $T // t_i$. Tổng này tăng theo T nên bài toán đơn điệu.

Gợi ý 3. Cận trên phải là một giá trị **chắc chắn đủ**. Máy nhanh nhất làm một mình toàn bộ m sản phẩm thì mất bao lâu?

Lời giải. Cận trên là $\min(t) \cdot m$, có thể tới 10^{18} .

```
lo = 1; hi = min(t) * m    # the fastest machine alone making every item
```

Đây là bài dạy về cận trên, và bản C++ nêu hai cái bẫy ngược nhau. Ở Python chỉ còn một.

- **Đặt cận trên bằng chính m.** Còn nguyên, và là cái bẫy chính. Rất nhiều người làm vậy theo phản xạ vì m là con số lớn duy nhất nhìn thấy trong đề. Nếu mọi máy đều chậm thì đáp án lớn hơn m nhiều lần, vòng chặt trả về đúng cận trên, và con số đó trông vẫn hợp lý. Ví dụ ngay trong đề bắt được: ba máy 3, 2, 5 với $m = 7$ cho đáp án 8, còn cận trên $m = 7$ thì đáp án in ra 7.
- **Hàm kiểm tự tràn nên phải thoát sớm. Không còn.** Bản C++ phải viết `if s >= m: return True` để tổng khỏi vượt 64 bit khi T cỡ 10^{18} và có máy `t_i = 1`. Số nguyên Python không tràn, nên dòng ấy chỉ còn là một tối ưu tốc độ. Và nó là một tối ưu **âm**: đo trên test lớn nhất, 321 ms có dòng thoát sớm so với 282 ms không có. Mục I.10 có bảng đủ tám bài, tám trên tám đều chậm đi.

Nếu bạn đọc bản C++ rồi chép dòng thoát sớm sang Python "cho chắc", bạn vừa trả 14 phần trăm thời gian cho một hiểm họa không tồn tại. Đó là cách một tối ưu đúng ở ngôn ngữ này thành một khoản lỗ ở ngôn ngữ kia.

Bài H. Phân tử thứ k của bảng nhân

Gợi ý 1. Bảng có tới $2,5 \cdot 10^9$ ô, không thể dựng ra rồi sắp xếp.

Gợi ý 2. Chặt nhị phân trên chính giá trị đáp án. Với giá trị x, đếm xem có bao nhiêu ô không vượt quá x.

Gợi ý 3. Ở dòng i, các ô là i, 2i, 3i, ..., mi. Số ô không vượt x là $\min(m, x // i)$.

Lời giải.

```
c = 0
for i in range(1, n + 1):
    d = mid // i
    c += d if d < m else m      # row i has only m cells, cap it
```

Độ phức tạp $O(n \log(n \cdot m))$.

Việc chặn số ô của một dòng lại là chỗ quyết định. Bỏ nó đi thì với i nhỏ và x lớn bạn đếm nhiều ô hơn số ô thật sự có trên dòng đó, số đếm phồng lên và đáp án nhỏ hơn thực tế. Ví dụ ba nhân ba của đề **vẫn cho đúng đáp án** kể cả khi bỏ, vì ở đó $x // i$ không bao giờ vượt 3; phải tới test 50.000 nhân 50.000 lỗi mới lộ. Đây là dạng lỗi tệ nhất: nó qua được mọi thứ bạn thử bằng tay.

Một điểm đáng nói: đáp án của vòng chặt ở đây **luôn là một giá trị có thật trong bảng**. Lý do: giá trị nhỏ nhất có số đếm đạt k không thể là một số vắng mặt, vì nếu x vắng mặt thì số đếm tại x bằng số đếm tại $x - 1$, và khi đó $x - 1$ đã đạt k rồi.

Python. Viết `c += d if d < m else m` chứ không `c += min(m, d)`. Biểu thức này chạy khoảng 1,6 triệu lượt trên test lớn nhất, và `min` là một lời gọi hàm thật chứ không phải một phép máy. Đây là bài dễ nhất buổi, 65 ms, nên khoản ấy không đổi kết luận gì; nhưng cùng một thói quen ở một bài nặng hơn thì đổi.

Bài I. Khoảng cách cặp nhỏ thứ k

Gợi ý 1. Số cặp là gần $2 \cdot 10^{10}$, không liệt kê được.

Gợi ý 2. Chặt nhị phân trên khoảng cách. Với d cho trước, đếm số cặp có khoảng cách không vượt d.

Gợi ý 3. Sau khi sắp xếp, việc đếm đó là một lượt hai con trỏ, đúng kỹ thuật của buổi trước.

Lời giải.

```
c = 0; l = 0
for r in range(n):
    xr = x[r]
    while xr - x[l] > mid:
        l += 1
    c += r - l      # every index in [l, r) pairs with r
```

Độ phức tạp $O(n \log n + n \log C)$, với C là bề rộng khoảng giá trị.

Hai chỗ dễ sai, và ở Python chỉ còn một:

- **Cộng thêm r trừ l , chứ không phải r trừ l cộng một.** Tức là $c += r - l$. Còn nguyên. Cộng thêm một là bạn đếm cả cặp gồm phần tử r với chính nó, số đếm phồng lên đúng n đơn vị và đáp án lệch xuống. Ví dụ của đề bắt được: đáp án thành 0 thay vì 4.
- **Biến đếm để kiểu `int`.** Không còn.

Và một chỗ chỉ Python mới có: **con trỏ trái không được đặt lại bên trong vòng lặp phải**, tức l nằm ngoài `for r`. Đặt lại thì lượt quét thành $O(n^2)$ và bài chết chắc. Đó chính là toàn bộ ý nghĩa của hai con trỏ.

Python, và đây là số đo bất ngờ nhất buổi. Việc đếm này viết được bằng `bisect_left(x, x[r] - mid, 0, r)`, tức là đẩy hẳn phần tìm kiếm xuống C . Kết quả ngược với trực giác:

Cách đếm	Thời gian
hai con trỏ, Python thuần	523 ms
<code>bisect_left</code> , một lời gọi mỗi phần tử	1.143 ms

Bản gọi thư viện chậm hơn 2,2 lần. Lý do: hai con trỏ trả giá **một bước trượt** được khấu hao cho mỗi phần tử, còn `bisect` trả giá **một lời gọi hàm Python đầy đủ** cho mỗi phần tử, và một lời gọi hàm đắt hơn một bước trượt nhiều lần. Ở bài A, một lời gọi `bisect_left` thay cho 18 bước lặp nên nó thắng đậm. Ở đây nó thay cho một bước, nên nó thua.

Quy tắc đáng mang theo: **hàm viết bằng C chỉ thắng khi một lời gọi của nó xoá được nhiều bước thông dịch.** Đếm xem nó xoá được mấy bước trước khi tin rằng nó nhanh hơn.

Bài J. Thêm trạm xăng

Gợi ý 1. Câu hỏi quyết định: với khoảng cách lớn nhất là d , cần thêm ít nhất bao nhiêu trạm?

Gợi ý 2. Mỗi khoảng trống dài g cần chia thành các đoạn không dài quá d , tức là cần $\text{ceil}(g / d)$ đoạn, tức là thêm $\text{ceil}(g / d) - 1$ trạm.

Gợi ý 3. Tổng số trạm cần thêm giảm khi d tăng, nên đây là mẫu tìm giá trị nhỏ nhất.

Lời giải. Sắp xếp, dựng sẵn mảng khoảng trống, rồi chặt trên d từ 1 tới khoảng trống lớn nhất.

```
s += (-g // mid) - 1 # ceiling, then one fewer station than pieces
```

Viết `g // mid - 1` là sai, và sai **theo hướng nguy hiểm**: nó báo cần ít trạm hơn thực tế, nên vòng chặt chấp nhận một d quá nhỏ và trả về đáp án nhỏ hơn đáp án đúng. Nhỏ hơn nghĩa là trông đẹp hơn, nên bạn càng khó nghi ngờ. Ví dụ của đề không bắt được lỗi này, vì ba khoảng trống đều dài 4 và chia hết cho 2; phải tới test khác mới lộ.

Dựng sẵn mảng khoảng trống. `gap = [x[i] - x[i - 1] for i in range(1, n)]` làm một lần, ngoài vòng chặt. Tính hiệu ngay trong hàm kiểm thì bạn trả cùng một phép trừ ấy 31 lần thay vì một lần.

Cũng như bài G, bản C++ cho hàm kiểm thoát sớm khi tổng đã vượt k . Ở đây lý do ấy cũng chỉ là tốc độ, và cũng là một khoản lỗ: 150 ms có thoát sớm so với 139 ms không có.

1. Ghi chú về bộ test kèm theo

Bộ test dùng chung với nhánh C++, nhánh này không thêm test riêng nào. Mười bài có tổng cộng 34 test, mỗi bài có một test cơ tối đa.

Cổng đột biến `tools/qa-dot-bien.py` tiêm **12 cái bẫy** vào mười bản mẫu đang AC, và bộ test bắt được **cả 12**: mười cái cho WA và hai cái cho TLE, là hai đột biến làm tròn sai chiều ở bài B và bài D. Không có bẫy nào sống sót, nên bộ test của buổi này không có nợ mở.

Đáng ghi lại một chi tiết: **bảng bẫy của nhánh Python không thể chép từ nhánh C++**. Nhánh C++ có một cổng riêng chuyển hạ `long` xuống `int`, và toàn bộ cổng ấy vô nghĩa ở đây. Ba chỗ mà tài liệu C++

ghi là "bộ test không bắt được lỗi tràn" cũng vậy: test 03 của bài F dựng riêng để ép tổng số giờ vượt 2^{32} , và ở Python nó không còn ép được gì. Nó vẫn nằm trong bộ test và vẫn chạy, chỉ là với tư cách một test lớn bình thường chứ không phải một test chuyên đề.

Phần IV. Mười code mẫu đầy đủ

Mười file này là **đúng** nội dung trong solutions/ref/, tức đúng thứ bộ chấm chạy. Công tools/qa-note-code.py so từng dòng và trượt nếu hai bên lệch nhau, vì mã in ra giấy không được bộ test nào chấm nên nó là chỗ dễ lạc hậu nhất trong cả kho.

Chú thích viết bằng tiếng Anh, chỉ ASCII. Không phải vì thắm mỹ: một chữ tiếng Việt có dấu lọt vào khối code làm XeLaTeX bỏ hẵn ký tự đó khỏi trang in, mà trang in vẫn trông bình thường.

Bài A. Đếm phần tử nhỏ hơn

```
# Problem A: count elements strictly less than x, binary search written by hand.
# The half open invariant [lo, hi) is the one worth memorising: lo is always a candidate
# answer, hi is always one past the range still under consideration, and the loop ends
# exactly when the window is empty.
#
# Python note. bisect.bisect_left(a, x) returns this very number at C speed: measured on
# the largest test, 109 ms against the 264 ms this hand written loop takes. The statement
# forbids it on purpose, because every later problem of the topic searches over an ANSWER
# RANGE with no array to hand, so the loop has to live in your fingers. Two more Python
# only points appear in this file:
# - (lo + hi) >> 1 is safe here. Python integers never overflow, so the C++ habit of
#   writing lo + (hi - lo) / 2 guards against a hazard that does not exist, and it is
#   not free: the same file measured 248 ms with >> 1, 262 ms with // 2 and 298 ms
#   with lo + (hi - lo) // 2. The form still matters for NEGATIVE bounds, because //
#   rounds toward minus infinity while C++ integer division rounds toward zero; lo and
#   hi are indices here, so they are never negative.
# - the comparison is <, not <=. With <= the loop computes bisect_right instead of
#   bisect_left, which answers a different question.
# Complexity: O((n + q) log n).
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); q = int(data[1])
    a = list(map(int, data[2:2 + n]))
    p = 2 + n

    out = []
    ghi = out.append
    for _ in range(q):
        x = int(data[p]); p += 1
        lo = 0; hi = n          # the answer lives in [lo, hi]
        while lo < hi:
            mid = (lo + hi) >> 1
            if a[mid] < x:
                lo = mid + 1     # mid is not the answer, discard it
            else:
                hi = mid         # mid may still be the answer, keep it
        ghi(lo)

    sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()
```

Bài B. Căn bậc hai nguyên

```

# Problem B: integer square root of n up to 1e18, without any floating point helper.
# This is the MAXIMUM pattern: find the largest r with r * r <= n. The midpoint must be
# rounded UP, otherwise lo = mid leaves lo unchanged when hi == lo + 1 and the loop
# never ends. The verdict is then TLE on a perfectly correct algorithm.
#
# Python note, and it differs from the C++ version of this topic on purpose.
# - math.sqrt is still wrong: it is a C double with 53 reliable bits and n needs 60.
#   But math.isqrt (3.8 and up) is EXACT and would solve the whole problem in 26 ms
#   against the 233 ms of this loop. The statement forbids both, because the point is
#   the loop.
# - "hi = n makes mid * mid overflow" is a C++ sentence. Python integers do not
#   overflow, so that bound is merely slower, not wrong: measured 428 ms against
#   233 ms, because it doubles the number of rounds and squares far larger numbers.
# Complexity: O(q log(hi)).
import sys

def main():
    data = sys.stdin.buffer.read().split()
    q = int(data[0])

    out = []
    ghi = out.append
    for i in range(1, q + 1):
        n = int(data[i])
        # The answer never exceeds 1e9, so 2e9 is a bound that certainly works and
        # keeps the loop to 31 rounds instead of 60.
        lo = 0; hi = 2000000000
        while lo < hi:
            mid = (lo + hi + 1) >> 1      # round UP, or lo = mid loops forever
            if mid * mid <= n:
                lo = mid
            else:
                hi = mid - 1
        ghi(lo)

    sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()

```

Bài C. Chia dãy thành k đoạn

```

# Problem C: split the array into exactly k consecutive parts, minimising the largest
# part sum. Do not search for the split. Ask the easier decision question instead: with
# a limit L, can the array be cut into at most k parts? That question is monotone, a
# larger L is always easier, so binary search on L.
# The check is greedy: keep filling the current part until it would pass L, then open a
# new one. That uses the fewest parts any split can use for that L.
#
# Python note, and it is a MEASURED correction of a habit borrowed from the segment tree
# topic. There the advice is to inline every helper, because a call happens per node. It
# does not transfer here: the check is called only about 48 times, once per round, so on
# the largest test this file takes 291 ms inlined and 289 ms with the check in a helper
# function. The check is kept inline only because the rest of the topic reads that way.
# An early break once cnt passes k was tried too, and it measured SLOWER, 306 ms against
# 292 ms: the test costs more on every element than it saves on the rare short circuit.
# Complexity: O(n log(sum)).

```

```

import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
    a = list(map(int, data[2:2 + n]))

    lo = max(a)          # one part must hold the largest element
    hi = sum(a)          # one single part holding everything always works
    while lo < hi:
        mid = (lo + hi) >> 1
        cnt = 1; cur = 0
        for v in a:
            if cur + v > mid:      # strictly greater: cur + v == mid still fits
                cnt += 1; cur = v
            else:
                cur += v
        if cnt <= k:
            hi = mid
        else:
            lo = mid + 1

    sys.stdout.write(str(lo) + '\n')

main()

```

Bài D. Đặt bò vào chuồng

```

# Problem D: place k cows in the given stalls so that the smallest gap between two cows
# is as large as possible. The monotone direction is the opposite of problem C: if a gap
# d can be achieved then every smaller gap can be too, so this is the MAXIMUM pattern
# and the midpoint rounds UP.
# The check is greedy: always take the leftmost stall, then take the next stall at least
# d away from the last one taken. That places as many cows as the gap d allows.
#
# Python note. This is the file that shows why the rounding rule survives the move to
# Python untouched: writing (lo + hi) >> 1 does not hang the machine in a visible way,
# it just spins, and the grader reports TLE on an algorithm that is completely correct.
# Nothing in a traceback points at the two bound lines, because there is no traceback.
# An early break once cnt reaches k measured 135 ms against 133 ms without, so it is not
# here; see the note in bai07.py for why the C++ habit does not carry over.
# Complexity: O(n log n + n log(range of coordinates)).
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
    x = sorted(map(int, data[2:2 + n]))

    lo = 0; hi = x[n - 1] - x[0]
    while lo < hi:
        mid = (lo + hi + 1) >> 1      # round UP, this is the max pattern
        cnt = 1; last = x[0]
        for v in x:
            if v - last >= mid:
                cnt += 1; last = v
        if cnt >= k:
            lo = mid
        else:
            hi = mid - 1
    sys.stdout.write(str(lo) + '\n')

main()

```

```

        lo = mid
    else:
        hi = mid - 1

    sys.stdout.write(str(lo) + '\n')

main()

```

Bài E. Cắt gỗ

```

# Problem E: set the saw at the highest integer height h that still yields at least M
# metres of wood. The yield only falls as h rises, so the predicate is monotone and this
# is again the MAXIMUM pattern, midpoint rounded up.
#
# The trap of this problem is not in the binary search at all, it is in the line that
# adds up the wood: a tree shorter than the blade contributes NOTHING, not a negative
# amount. Drop the guard and short trees eat away at the total.
#
# Python note. The sum reaches 2e14. In C++ that forces long long and forgetting it is
# a real defect; in Python the integer simply grows and nothing is lost, so that whole
# warning disappears from this topic. There is no early break either: stopping the sweep
# as soon as s reaches M measured SLOWER on the largest test, 172 ms against 162 ms.
# Complexity: O(n log(tallest tree)).
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); M = int(data[1])
    a = list(map(int, data[2:2 + n]))

    lo = 0; hi = max(a)          # a blade above the tallest tree yields nothing
    while lo < hi:
        mid = (lo + hi + 1) >> 1
        s = 0
        for v in a:
            if v > mid:          # the guard is the whole point of this problem
                s += v - mid
        if s >= M:
            lo = mid
        else:
            hi = mid - 1

    sys.stdout.write(str(lo) + '\n')

main()

```

Bài F. Tốc độ ăn chuối

```

# Problem F: smallest eating speed v such that the total number of hours fits in h.
# One pile of a bananas takes the CEILING of a / v hours, because the last hour counts
# in full even when the pile runs out early. Total hours fall as v grows, so this is the
# MINIMUM pattern and the midpoint rounds down.
#

```

```

# Python note on the ceiling. Three ways to write it, and only two are safe:
# -(-a // v)          exact for every integer, and it is the idiom to memorise
# (a + v - 1) // v    exact while a and v are positive, the C++ habit
# math.ceil(a / v)    WRONG above 2**53, because a / v is a float first
# The first two measured the same, 6 ms per 200000 divisions, so pick either. The third
# is not a speed question: math.ceil(10**18 / 3) gives 33333333333333312 where the true
# ceiling is 33333333333333334, and 10**400 / 3 raises OverflowError outright.
# Python // rounds toward minus infinity, which is exactly what makes the first form
# work; C++ integer division truncates toward zero and the same expression fails there
# for negative a.
# No early break here either: stopping as soon as s passes h measured 208 ms against
# 171 ms without.
# Complexity: O(n log(largest pile)).
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); h = int(data[1])
    a = list(map(int, data[2:2 + n]))

    lo = 1; hi = max(a)          # speed equal to the largest pile needs n hours
    while lo < hi:
        mid = (lo + hi) >> 1
        s = 0
        for v in a:
            s += -(-v // mid)    # ceiling division, never a plain //
        if s <= h:
            hi = mid
        else:
            lo = mid + 1

    sys.stdout.write(str(lo) + '\n')

main()

```

Bài G. Máy sản xuất

```

# Problem G: smallest number of minutes T in which k machines build at least m items.
# In T minutes machine i builds T // t[i] items, and that total only grows with T, so
# the predicate is monotone and this is the MINIMUM pattern.
#
# This is the upper bound problem of the topic. Setting hi = m is the reflex, because m
# is the only large number in the statement, and it is wrong: if every machine is slow
# the answer is many times m. A binary search over a range that misses the answer does
# not crash, it quietly returns its own upper bound, and that number looks plausible.
# The bound that certainly works is the fastest machine building all m items alone.
#
# Python note, and this is the file where the C++ advice reverses. The C++ version breaks
# out of the accumulation loop the moment s reaches m, because otherwise the sum passes
# 64 bits and the check silently lies. Python integers never overflow, so that reason is
# gone. Copying the break anyway is not free insurance: measured on the largest test it
# costs time rather than saving it, 321 ms with the break against 282 ms without, since
# the extra comparison runs on every one of the 2e5 machines in every round.
# Complexity: O(k log(m * smallest t)).
import sys

def main():

```

```

data = sys.stdin.buffer.read().split()
k = int(data[0]); m = int(data[1])
t = list(map(int, data[2:2 + k]))

lo = 1; hi = min(t) * m      # the fastest machine alone making every item
while lo < hi:
    mid = (lo + hi) >> 1
    s = 0
    for v in t:
        s += mid // v
    if s >= m:
        hi = mid
    else:
        lo = mid + 1

sys.stdout.write(str(lo) + '\n')

main()

```

Bài H. Phần tử thứ k của bảng nhân

```

# Problem H: k-th smallest value in the n by m multiplication table.
# The table has up to 2.5e9 cells, so it cannot be built. Binary search on the VALUE
# instead: the number of cells whose value is at most x is the sum over rows i of
# min(m, x // i), and that count only grows with x. The smallest x whose count reaches
# k is the answer, and it is always a real entry of the table: if x were absent then the
# count at x equals the count at x - 1, so x - 1 would already have reached k.
#
# The min with m is the line that decides correctness. Row i holds only m cells, so
# x // i must be capped; drop the cap and the count inflates for small i, which drags
# the answer below the true one. A 3 by 3 sample still passes, so the defect lives on
# until a large test.
#
# Python note. min(m, d) is a function call, and this expression runs about 1.6e6 times
# on the largest test, so the conditional expression below does the same job without the
# call. An early break once c reaches k was measured and dropped: 70 ms with it against
# 61 ms without.
# Complexity: O(n log(n * m)).
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1]); k = int(data[2])

    lo = 1; hi = n * m
    while lo < hi:
        mid = (lo + hi) >> 1
        c = 0
        for i in range(1, n + 1):
            d = mid // i
            c += d if d < m else m      # row i has only m cells, cap it
        if c >= k:
            hi = mid
        else:
            lo = mid + 1

    sys.stdout.write(str(lo) + '\n')

```

```
main()
```

Bài I. Khoảng cách cặp nhỏ thứ k

```
# Problem I: k-th smallest distance among all pairs of points.
# There are up to 2e10 pairs, so they cannot be listed. Binary search on the distance,
# and for a candidate d count the pairs no further apart than d. After sorting, that
# count is a two pointer sweep: for each right end r, slide l forward while the window
# is too wide, then every index in [l, r) pairs with r. This is where the two pointer
# topic and the binary search topic meet, and a great many hard problems have exactly
# this shape.
#
# c += r - l, not r - l + 1. The extra one counts the pair of r with itself, which
# inflates the count by exactly n and pushes the answer down.
#
# Python note. l is NOT reset inside the r loop, so the sweep is O(n) per check rather
# than O(n^2); that is the whole point of two pointers. The tempting alternative is
# bisect.bisect_left(x, x[r] - mid, 0, r), which runs in C. It measured WORSE on the
# largest test, 1143 ms against 523 ms, because it pays one Python level function call
# per element while the sweep pays an amortised single step. A library routine written
# in C only wins when one call replaces many interpreted steps, not when it replaces one.
# Complexity: O(n log n + n log(range of coordinates)).
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
    x = sorted(map(int, data[2:2 + n]))

    lo = 0; hi = x[n - 1] - x[0]
    while lo < hi:
        mid = (lo + hi) >> 1
        c = 0; l = 0
        for r in range(n):
            xr = x[r]
            while xr - x[l] > mid:
                l += 1
            c += r - l          # every index in [l, r) pairs with r
        if c >= k:
            hi = mid
        else:
            lo = mid + 1

    sys.stdout.write(str(lo) + '\n')

main()
```

Bài J. Thêm trạm xăng

```
# Problem J: add at most k new stations so that the largest gap between neighbouring
# stations is as small as possible. Decision question: with a largest gap of d, how many
# stations must be added? A stretch of length g has to be cut into pieces no longer than
# d, which takes ceil(g / d) pieces and therefore ceil(g / d) - 1 new stations. The
```

```

# total only falls as d grows, so this is the MINIMUM pattern.
#
# Writing g // d - 1 instead of the ceiling is wrong in the dangerous direction: it
# reports fewer stations than reality, so the search accepts a d that is too small and
# prints an answer BELOW the true one. Smaller looks better, so it invites less suspicion.
#
# Python note. -(-g // d) is the ceiling, because // rounds toward minus infinity. The
# C++ form (g + d - 1) // d is written that way to dodge an overflow Python does not
# have; both are exact for positive g and d, both measure the same, and both are safe
# here. math.ceil(g / d) is not: it rounds a float that was already rounded.
# The C++ version stops the sweep early once s passes k. That break measured 150 ms
# against 139 ms without it, so it is gone.
# Complexity: O(n log n + n log(widest gap)).
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
    x = sorted(map(int, data[2:2 + n]))

    gap = [x[i] - x[i - 1] for i in range(1, n)]
    lo = 1; hi = max(gap)
    while lo < hi:
        mid = (lo + hi) >> 1
        s = 0
        for g in gap:
            s += -(-g // mid) - 1      # ceiling, then one fewer station than pieces
        if s <= k:
            hi = mid
        else:
            lo = mid + 1

    sys.stdout.write(str(lo) + '\n')

main()

```